

Agent 开发周报 260208

刘德辰

2026 年 02 月 08 日

1 规则驱动智能体（Rule-Driven Agent）核心功能实现

本周重点完成了规则驱动智能体的核心架构与功能实现，该系统允许客户通过自然语言配置回复规则，系统在运行时根据 Router 的匹配结果自动按规则生成稳定且自然的回复。

1.1 架构设计说明

本系统的架构设计采用了分层设计思路：

- **总体架构：**基于 RAG（Retrieval-Augmented Generation）通用方案，采用向量检索 + LLM 生成的经典架构模式。
- **Config 架构：**针对规则配置场景，设计了”双轨交付”和”对话式创建”架构（Ask / Build / Reply 三类技能），该设计来源于对用户需求和场景的深入分析，通过对话式配置降低客户使用门槛，通过编译机制规范规则格式，保证规则的可执行性和安全性，是结合了业务场景的创新设计。

1.2 Config 架构与交互模式设计（核心亮点）

1.2.1 对话式创建：降低配置门槛的核心创新

系统的核心亮点是**对话式创建机制**，让客户用”人话”配置规则，无需理解技术概念。

- **Rule Asker（规则配置追问向导）：**采用”需求访谈”式的对话方式，像与产品经理沟通一样自然
 - 通过自然语言逐步追问收集规则信息（规则名称、触发描述、回复目标、关键要点、必填信息、模板、禁区、语调等）
 - 每轮最多追问 1-2 个问题，避免人脑信息过载，降低配置门槛
 - 当信息齐全时输出确认摘要，引导用户确认保存
 - 支流式输出（SSE），提供实时的交互反馈
 - 产物为 `rule_draft`（草稿结构），采用业务语言表达，存储在 `rule_drafts` 表中
- **Rule Builder（规则编译器）：**分析草稿和 asker 对话全文，将其编译为规范化规则格式
 - 补全默认值、规范化字段、生成规则 ID、注入安全底线
 - 使用 embedding service 生成向量，用于后续规则匹配
 - 输出严格的 JSON 格式，可校验、可存储、可版本化
 - 保存前自动进行冲突检测，防止规则重复或语义冲突

1.2.2 双轨交付：交互与编译的物理隔离

系统延续”双轨”理念，将交互式对话与结构化编译物理隔离，实现配置体验与系统可靠性的最佳平衡：

- **交互式轨道：**Rule Asker 和 Rule Reply 采用 Markdown 流式输出，提供流畅的用户体验
- **结构化轨道：**Rule Builder 输出 Strict JSON，经过兜底处理和 check 后保存，保证规则格式的规范性和执行稳定性

1.3 Embedding Service 层（规则匹配服务）

- **独立匹配服务：**新增了 `match_rules` 工具，作为 Router Pipeline 中的独立服务层，专门负责规则的向量化与相似度匹配。该服务不依赖 Router LLM 的判断，使用专门的 Embedding 模型进行向量化计算。
- **混合检索策略：**采用 **Anchor + Examples** 的混合检索策略，通过 `match_text`（锚点）保证召回率，通过 `examples`（用户真实语料示例）提升精确率和排序质量。融合算法使用 Max Fusion 策略，取锚点得分和示例最大得分的较高值作为最终得分。
- **向量化实现：**使用阿里云 QWEN `text-embedding-v1` 模型进行向量化，支持规则保存时自动生成 embedding。向量数据存储在 `rules.embedding` 字段中，采用 JSON 格式存储锚点和示例的向量数组。
- **当前实现方式：**受限于当前测试 demo 环境后端部署在 Cloudflare Workers 和 D1 数据库上，当前实现采用**全量加载 + 内存计算**的方式：
 - 从 D1 数据库全量加载所有启用规则（当前限制最多 500 条）
 - 在 Workers 内存中对每个规则计算余弦相似度
 - 按相似度排序并返回 TopK 结果
 - **限制：**D1 (SQLite) 不支持向量索引，无法进行高效的向量检索；Workers 有执行时间限制（通常 30 秒），当规则数量增长时性能会急剧下降
- **向量数据库未实现：**由于上述环境限制，当前**未实现向量数据库**（如 Milvus、Qdrant、Pinecone 等），向量检索相关的高效索引和检索功能尚未实现。这是当前架构的主要技术债务，生产环境中会引入专门的向量数据库来解决该问题。
- **TopK 检索：**实现了相似度计算和排序算法，支持自定义 `top_k`（默认 5）和 `min_score`（默认 0.3）参数，返回匹配度最高的规则列表。

1.4 Agent 技能（Ask / Build / Reply）

系统新增三个技能，严格遵循”双轨交付”理念，实现了规则配置、编译和执行的完整闭环。

- **Rule Asker（规则配置追问向导）：**面向客户的对话式配置 Agent，通过自然语言逐步追问收集规则信息。每轮最多追问 1-2 个问题，收集包括规则名称、触发描述、回复目标、关键点、必填信息、模板、禁区、语调等字段。当信息齐全时输出确认摘要，引导用户确认保存。产物为 `rule_draft`（草稿结构），存储在 `rule_drafts` 表中。

- **Rule Builder (规则编译器)**: 面向系统的结构化编译器 Agent, 负责将 `rule_draft` 编译为 `rule_json`。职责包括补全默认值、规范化字段、生成规则 ID、注入安全底线、生成 embedding 向量。输出为严格的 JSON 格式, 可校验、可存储、可版本化。
- **Rule Reply (规则执行器)**: 面向运行时的规则执行工具, Router 已提供命中规则 IDs 和匹配分数。负责读取规则详情, 按规则模板生成最终回复 (Markdown 流式输出)。支持模板填空、缺参追问、安全底线检查等功能。

1.5 Router 集成与决策流程

- **Router 工具扩展**: 在 Router 的可用工具中新增 `match_rules` 工具, Router LLM 可以自主决定是否调用该工具进行规则匹配。
- **决策逻辑优化**: Router 根据 `match_rules` 返回的匹配结果进行决策:
 - 如果 top1 匹配分数 > 0.7 $\rightarrow$ 派单 `rule_reply`, 传入 `hit_rules` 和 `user_query`
 - 如果匹配分数较低或无匹配 $\rightarrow$ 继续走其他 Worker (如 `consult_omni`) 或拒绝
- **场景表兼容**: 规则匹配机制与现有的工作场景表机制兼容, Router 可以结合场景表和规则匹配结果做出最终决策。
- **决定因素**: 任务分发仍由 Router agent 最终决定, 匹配结果为参考因素。

2 规则冲突检测机制

实现了规则保存前的冲突检测功能, 防止规则重复或语义冲突。

- **自检索机制**: 在 `rule_builder` 保存规则前, 对当前草稿的 `match_text` 生成 Embedding 向量, 并在规则库中执行 Top-K 检索 (排除当前规则 ID)。
- **三级阈值判定**:
 - 红区 (> 0.92): 高度冲突, 视为重复规则, 阻断保存, 提示用户直接修改旧规则
 - 黄区 ($0.80 - 0.92$): 潜在冲突, 语义相近, 提示用户建议合并, 询问是否强制保存
 - 绿区 (< 0.80): 安全, 直接保存
- **冲突检测工具**: 实现了 `detectRuleConflict` 函数, 支持在规则新建和编辑时进行冲突检测, 确保规则库的质量和一致性。

3 数据库架构扩展

新增了规则系统的数据库表结构, 支持规则和草稿的存储与管理。

- **rules 表**: 存储编译后的可执行规则, 包含 `id`、`name`、`match_text`、`enabled`、`priority`、`version`、`embedding`、`data` (规则 JSON 数据) 等字段。建立了 `enabled` 和 `priority` 索引, 支持规则的启用/禁用和优先级管理。
- **rule_drafts 表**: 存储规则配置过程中的草稿数据, 包含 `session_id` (主键)、`status`、`mode` (`create/edit`)、`rule_id`、`draft` (草稿 JSON 数据) 等字段。支持多会话并发配置规则。

4 规则数据结构设计

设计了 `rule_draft` (草稿) 和 `rule_json` (可执行规则) 两阶段数据模型。`rule_draft` 采用业务语言字段 (`name`、`match_text`、`examples`、`reply_goal`、`key_points`、`required_info`、`template`、`safe_defaults`、`do_not_say`、`tone` 等), 降低配置门槛; `rule_json` 为系统内部严格 JSON 格式, 包含完整执行逻辑和 `embedding` 字段。

5 系统架构优化

延续 Router 与 Worker 职责分离架构, 规则工具通过统一接口接入系统, `rule_asker` 和 `rule_reply` 支持流式输出 (SSE)。

6 文档与规范

完成《规则列表系统开发文档.md》和三类技能的 `SKILL.md` 文档, 在关键代码中添加详细注释。

7 技术债务与限制

向量数据库未实现: 受限于 Cloudflare Workers + D1 环境, 当前采用全量加载 + 内存计算方式, 无法支持大规模规则库的高效检索。D1 不支持向量索引, Workers 有执行时间和内存限制, 当规则数量超过数百条时性能会急剧下降。需引入专门的向量数据库解决方案。

8 下周计划

1. **向量数据库集成:** 调研并实现向量数据库方案, 重构 `match_rules` 和 `detectRuleConflict` 使用向量数据库进行高效检索。
2. **规则测试功能:** 实现规则测试工具, 支持保存前测试匹配效果和回复质量。
3. **规则版本管理:** 完善版本控制机制, 支持历史版本查看和回滚。
4. **规则模板库:** 建立常用规则模板库, 降低配置成本。